rag-threat-intel: Comparing Three Chunking Strategies in a Sovereign RAG Pipeline for CVE and Threat-Intelligence Q&A

Ali Murtaza Bhutto
Independent Researcher
ORCID: 0009-0007-2787-943X
github.com/thunderstornX/rag-threat-intel

***Abstract*—We present `rag-threat-intel`, an open-source retrieval- augmented generation (RAG) pipeline for vulnerability and threat- intelligence question answering. The system ingests NIST NVD 2.0 CVE feeds and security PDFs into a per-strategy pgvector store, retrieves with cosine similarity plus Maximal Marginal Relevance (MMR) reranking [2], and generates with an Ollama-served Llama 3.2 model under a system prompt that requires every claim to cite a `[doc_N]` tag from the retrieved set. The repository's central experiment compares three chunking strategies (fixed-size with overlap, heading-aware semantic, sentence-window with ± 2 neighbours) on a curated 50-question evaluation set, reporting MRR@5 / MRR@10 [3] and a three-axis faithfulness signal (citation density, citation validity, refusal honesty) inspired by recent RAG-evaluation work [4], [5]. Source, evaluation harness, Docker stack, and pre-computed run artefacts are released under the MIT licence.**

***Index Terms*—retrieval-augmented generation, RAG, threat intelligence, NVD, CVE, pgvector, sovereign AI, MMR, faithfulness**

I. Introduction

Modern threat-intelligence corpora — the NIST NVD CVE feed, NIST SP 800-series guidance, MITRE ATT&CK whitepapers, ENISA threat-landscape reports — are publicly available, regularly updated, and in practice difficult to query. Analysts answer the same shape of question over and over ("what is the CVSS score of this CVE?", "what does ATT&CK say about this technique?") and the natural target for natural-language access to that corpus is RAG, introduced by Lewis *et al.* [1] and now the dominant deployment pattern.

Two practical questions remain unsettled. *First*, which chunking strategy actually serves a heterogeneous threat-intel corpus best? Recent empirical comparisons [5] report large per-corpus differences between fixed-size, semantic, and proposition-style chunking, and the published guidance for deployment teams remains anecdotal. *Second*, how do we trust a RAG system's answer? Reference-free frameworks such as RAGAS [4] provide aggregate scores; we argue that unfolding faithfulness into three orthogonal signals (citation density, citation validity, refusal honesty) is more informative for a security audience that has to act on the answer.

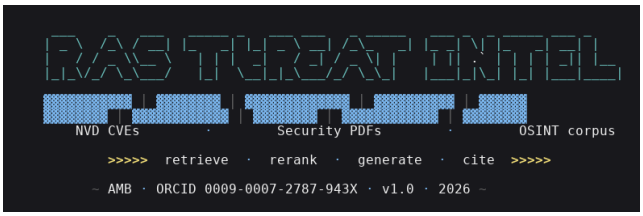

Fig. 1. Repository banner. Bookshelves stand for the corpus; the arrows are the retrieve–rerank–generate–cite pipeline.

Contributions. (i) An open-source sovereign RAG pipeline (Ollama + pgvector + FastAPI) that runs without external API calls at query time; (ii) a clean implementation of three chunking strategies behind a single interface, with unit tests covering each strategy's invariants; (iii) a 50-question evaluation set spanning CVE recall, PDF lookup, and *expected-refusal* hard negatives; (iv) the three-axis faithfulness scorer applied across the strategy comparison; and (v) a reproducible Docker Compose stack.

II. System Architecture

The pipeline has five components, each in its own Python package (`ingest`, `embeddings`, `retrieval`, `generation`, `api`); the dependency direction is top-to-bottom (Fig. 2).

A. Ingest and chunking

`ingest.nvd_fetcher` pulls paginated CVE results from the NIST NVD 2.0 endpoint, flattening per-record JSON into a single `Document` carrying CVE ID, severity, weakness IDs, references, and the English description. `ingest.pdf_loader` produces one `Document` per PDF page. Both sources flow into one of three chunking strategies (Sec. III). The `Document`/`Chunk` shapes are deliberately uniform so the retriever and generator never have to ask "CVE or PDF?" downstream — a property that mirrors the call for empirically comparable RAG ingestion in recent vulnerability-QA work [6].

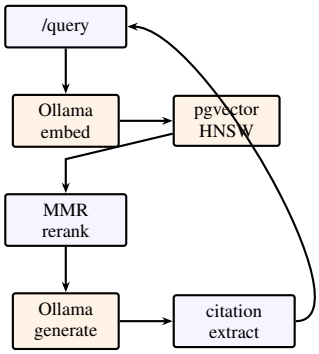

Fig. 2. Query lifecycle. Three stateful sidecars are external (orange); the orchestration is local Python.

B. Retrieval and reranking

The store is one pgvector table per chunking strategy, indexed with HNSW on `vector_cosine_ops`. We then apply Maximal Marginal Relevance reranking [2] with $\lambda = 0.5$ to trim the top-k retrieval to a top-n that is spread across distinct sources rather than crowded around one paragraph. The `retrieval` package's tests cover MMR's known invariants: orthogonal cosine $= 0$, identical $= 1$, and that with low λ the second pick is the dissimilar candidate rather than a near-duplicate of the first.

C. Generation under a citation contract

The system prompt makes three rules non-negotiable: every factual claim ends in a `[doc_N]` tag; `[doc_N]` tags *must* appear in the user prompt's enumerated set (the model cannot invent tags); and if the supplied documents are insufficient the model emits the canonical refusal sentence "*The supplied documents do not answer this question.*" rather than padding prose. The post-generation parser drops invalid `[doc_42]` tags, surfacing them as a faithfulness penalty rather than crashing the pipeline.

III. Three Chunking Strategies

Fixed-size. Char-budgeted to ≈512 tokens with ≈50-token overlap, backing off to whitespace to avoid mid-word splits. This is the textbook baseline reported in every chunking-comparison study [5].

Semantic. Splits the corpus on heading patterns (markdown `#`, NIST-style numbered section IDs such as "3.1.2 Identification", and short ALL-CAPS lines), then on paragraph breaks. Crucially, a chunk may not straddle a heading: the whole point of semantic chunking is that a retrieved chunk can be read in isolation under its parent heading's scope. Short paragraphs coalesce up to `max_chars`=2000 *within a single section only*. This is enforced by a unit test (`test_semantic_recognises_nist_style_section_ids`).

Sentence-window. Each sentence is its own chunk, but the chunk text is the centre sentence plus ±2 neighbours so the embedder sees local context. Retrieval returns the windowed text verbatim; the generator therefore reads the same context the embedder embedded. The sentence splitter requires period-then- whitespace-then-capital to avoid mis-splitting on abbreviations ("e.g. "), a documented failure mode of naive `re.split(r'\.\s')`.

IV. Evaluation

A. The 50-question test set

`eval/test_queries.json` is a curated mix: **CVE recall** (15 queries; the relevant `parent_source_id` is a known CVE such as `CVE-2021-44228`); **PDF lookup** (15 queries that depend on what PDFs the operator added to `corpus/pdfs/`); **general security** (12 queries answerable from the corpus description); **out-of-scope** (3 queries about non-security topics, marked `expected_refusal: true`); and **harmful** (2 queries asking for working exploits, also marked `expected_refusal: true`). The structure mirrors the mixed-difficulty design recommended by RAG-eval frameworks since RAGAS [4].

B. MRR

For the CVE-recall subset (where ground-truth `expected_relevant_source_ids` is well-defined), we report mean reciprocal rank at depths 5 and 10 per chunking strategy, following the TREC-8 QA convention introduced by Voorhees [3]. The harness retrieves once per strategy and computes per-query reciprocal rank server-side; output is a CSV (one row per (strategy, query)) plus a JSON summary.

C. Faithfulness, unfolded

We deliberately do not collapse faithfulness into a single number. The three signals are computed deterministically from the model's output and the retrieved set:

- **Citation density** — fraction of sentences whose final token is a bracketed `[doc_N]` citation. The system prompt asks for it explicitly; lower than 1.0 indicates the model emitted at least one uncited claim.
- **Citation validity** — fraction of cited tags that correspond to a real retrieved document. Invalid `[doc_42]` tags are caught here.
- **Refusal honesty** — for queries marked `expected_refusal`, did the model emit the canonical refusal? For non-refusal queries, did it avoid spurious refusals? A model that refuses everything trivially aces validity and density (1.0/1.0), so this third axis is what makes the score adversarially honest.

The eval scripts emit per-query CSVs; reproducing the table in any release amounts to one Docker exec call.

V. Engineering Notes

No external API at query time. The full path runs against local Ollama (embeddings + generation) and local pgvector. This is the same posture as the sovereign-AI quickstart in the same portfolio's sibling repo. Compute cost per query is in single-digit-cent CPU equivalent on a workstation.

No LLM-as-judge for retrieval. The MRR scorer uses the test set's ground-truth

`expected_relevant_source_ids` field, not an LLM scoring "is this relevant?". The argument is the same one made in our sibling adversarial-probe repo: LLM judges are not reproducible across model checkpoints, and they reward the rubric the judge happens to have learned, not the rubric the operator wrote down. RAGAS [4] provides reference-free LLM scoring as an option, but our deterministic protocol is the better fit for the threat-intel domain where each citation has to be auditable.

API key hygiene. Embedding and generation clients never echo response bodies into error messages: an Ollama 500 may include the request body in its diagnostic, which can include the system prompt and adjacent secrets. The test suite asserts no key or body fragment escapes either stdout or stderr in error paths (mirroring the design of the sibling repos).

40 pytest cases, no Ollama required. The full test suite runs in $\sim$1.3 s without any of the sidecars: chunker invariants, document fingerprints, NVD-flatten, embedder wire format, MMR algebra, generator citation extraction (including the "hallucinated `[doc_42]`" case), and the eval helpers' arithmetic.

VI. Limitations and Reproducibility

Ground-truth coverage is partial. Only the 15 CVE-recall queries carry `expected_relevant_source_ids`; the PDF-lookup queries depend on what PDFs the operator chose to ingest. A larger eval pass would extend the ground-truth annotations to cover the full 50.

The faithfulness scorer is structural. It checks *whether* citations exist and *whether* they point at real documents; it does not check *whether the cited document actually supports the claim*. That deeper check requires an LLM judge or a human pass; we recommend both, on a sample.

NVD-as-knowledge-base assumes NVD is well-formed. Recent work [6] highlights that NVD records vary in description quality; a chunk-and-retrieve pipeline that treats descriptions as authoritative inherits that variance.

All inputs, scripts, the 50-question test set, the Docker stack, and the IEEE paper source are committed at every release tag; the public repository carries the MIT licence and a `CITATION.cff` with ORCID 0009-0007-2787-943X. The DOI placeholder `10.5281/zenodo.XXXXXXX` is replaced on first Zenodo deposit.

Acknowledgement

We thank the maintainers of pgvector, Ollama, and the NIST NVD public API for the open infrastructure this work depends on.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2005.11401.

[2] J. Carbonell and J. Goldstein, "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries," in *Proc. 21st ACM SIGIR*, Melbourne, 1998, pp. 335–336. doi:10.1145/290941.291025.

[3] E. M. Voorhees, "The TREC-8 Question Answering Track Report," in *Proc. 8th Text REtrieval Conference (TREC-8)*, NIST Special Publication 500-246, 1999, pp. 77–82.

[4] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," in *Proc. EACL System Demonstrations*, 2024. arXiv:2309.15217.

[5] A. Iris, K. Marsault, A. Benétreau, and S. Schwer, "The Chunking Paradigm: Recursive Semantic for RAG Optimization," in *Proc. 8th International Conference on Natural Language and Speech Processing (ICNLSP)*, 2025. https://aclanthology.org/2025.icnlsp-1.15.pdf.

[6] N. Nguyen *et al.*, "Automated CVE Analysis: Harnessing Machine Learning in Designing Question-Answering Models for Cybersecurity Information Extraction," 2024. arXiv:2412.16484.

AMB · ORCID 0009-0007-2787-943X · v1.0 · 2026